

SUPRAV CHAND

☎ +977 9818363103 ◇ Kathmandu, Nepal

✉ supravchand2@gmail.com [in LinkedIn](#) [GitHub](#)

WORK EXPERIENCE

Full Stack Developer Intern

Kathmandu Living Lab, Kathmandu, Nepal

March 2025 – June 2025

SEO Content Writer

Nepaya Solutions, Kathmandu, Nepal

November 2024 – January 2025

EDUCATION

Sagarmatha College Of Science And Technology (SCST)

Bachelor of Science in Computer Science and Information Technology

Aug 2021 - Jun 2025

Uniglobe SS/College

Higher Secondary

CGPA: 3.27/4

2018 - 2020

PROJECTS

Hospital Management System [🔗](#)

September 2024 to January 2025

- Focused on improving the Hospital management system to streamline appointment scheduling, patient management, and symptom analysis.
- Designed a User portal where the users personal details, appointment status is shown along with systems they searched while they were logged in the system and are displayed in the user portal.
- Patients appointment are only handled in FCFS (First Come First Serve order) based on status change done by Admin and the system has payment gateway which is required to book an appointment.
- Applied JWT authentication for secure user access and role-based authorization in both user and admin portals.
- **Tools used:** Python (scikit learn), React, Node.JS, Mongo DB, Esewa Gateway, Express JS.

Football Player Scouting System [🔗](#) [🔗](#)

February 2025 - March 2025

- Built an end-to-end football player scouting web application that recommends similar players using cosine similarity and clustering on real player statistics.
- Performed full data preprocessing pipeline on Kaggle cleaned dataset, engineered features, and trained similarity and clustering models using scikit-learn.
- Implemented advanced player filtering by position, age, and performance stats with interactive data visualizations including radar charts and comparison charts.
- Applied t-SNE dimensionality reduction to visualize player clusters in 2D, revealing natural groupings across positions and playing styles using Plotly interactive charts.
- Deployed as a Flask web application on Render with a responsive UI allowing scouts to search, compare, and analyze players side by side.
- **Tools used:** Python, scikit-learn, Flask, Pandas, Kaggle, HTML/CSS.

Nepali Legal Document Assistant [🔗](#) [🔗](#)

July 2026

- Built a RAG based legal document assistant that analyzes uploaded Nepali legal documents including rent agreements, court notices, and property documents to generate plain-language summaries, extract key clauses, and flag risky terms with severity levels.
- Implemented a hybrid document ingestion pipeline that automatically detects and handles both text-based and scanned PDFs falling back to OCR for image-based documents to extract text from low quality scans and photographs.

- Designed structured output chains using Pydantic schemas to extract typed clause objects and risk flags with low, medium, and high severity levels, enabling color coded visual display in the UI.
- Built a persistent multi document vector store supporting incremental PDF uploads, allowing users to upload multiple legal documents and query across all of them simultaneously.
- Deployed an interactive web application with real-time document processing, color coded risk visualization, expandable clause cards, and a multi turn chat interface for document Question and Answer.
- **Tools used:** Python, LangChain, ChromaDB, HuggingFace Embeddings, Tesseract OCR, Streamlit, Pydantic.

CV and Job Match Assistant

July 2026

- Built a Retrieval Augmented Generation (RAG) application that analyzes a candidate's CV against any job description to generate match scores, tailored cover letters, and skill gap analysis.
- Implemented a document ingestion pipeline using PyPDFLoader and Recursive Character TextSplitter, embedding CV chunks with HuggingFace's all MiniLM L6 v2 model and storing them in a persistent ChromaDB vector store.
- Designed a semantic retrieval system where job description queries dynamically fetch the most relevant CV sections, reducing irrelevant context and improving LLM output quality.
- Built structured-output LLM chains using LangChain and Groq's Llama 3.3 70B model with Pydantic schemas to generate consistent, machine-readable match scores and skill gap reports.
- **Tools used:** Python, LangChain, LangChain-Groq, ChromaDB, HuggingFace Embeddings, Streamlit, Pydantic.

Heart Disease Prediction System

April 2025 - May 2025

- Built an end-to-end heart disease prediction web application using two ML models Random Forest (87% accuracy) and PyTorch Neural Network (90% accuracy) trained on real clinical data.
- Performed full data preprocessing pipeline on Kaggle handled missing values, fixed impossible zeros using median imputation, applied Label Encoding and One Hot Encoding, and scaled features using StandardScaler.
- Trained and compared both models with accuracy, confusion matrix, and classification report, showing Neural Network outperforming Random Forest on test data.
- Deployed as a Flask web application where users input medical details (age, cholesterol, blood pressure etc.) and receive dual-model predictions with confidence scores side by side.
- **Tools used:** Python, PyTorch, scikit-learn, Flask, Pandas, Kaggle, HTML/CSS.

Urban Sound Audio Classifier

May 2025 - June 2025

- Built an audio classification system trained on the UrbanSound8K dataset (8,732 real-world audio clips across 10 categories sirens, drilling, dog barks, street music and more).
- Extracted rich audio features using Librosa including MFCCs, Mel spectrograms, chroma features, and spectral contrast converting raw audio signals into structured numerical representations for ML.
- Performed full preprocessing pipeline handled class imbalance, normalized features, and engineered a robust feature matrix from 6GB of raw audio data.
- Trained and evaluated classification model capable of distinguishing between 10 distinct urban sound categories with strong accuracy across unseen audio clips.
- **Tools used:** Python, Librosa, scikit-learn, NumPy, PyTorch, Pandas, Kaggle.

SKILLS

- **Languages:** JavaScript, Python
- **ML & AI:** PyTorch, scikit-learn, HuggingFace Transformers, Pandas, NumPy, Librosa
- **LLM & RAG:** LangChain, Groq API, ChromaDB, HuggingFace Embeddings, Prompt Engineering
- **Frameworks:** React, Node.JS, Express.JS, Flask
- **Database:** MongoDB, MySQL
- **Tools:** Git, Kaggle, Google Colab